Database Management System (303105203)

Unit – 5: Relational Database Design

Dr. Dhanraj Verma,
Professor, CSE-AIML

Unit Required

- Functional Dependency
- Closure of FD
- Closure of Attribute
- Extraneous Attributes
- Canonical Cover
- Finding a Candidate Key
- Decomposition
- Database Anomalies
- Normal Forms / Normalization

Functional Dependency

Student

RollNo	Name	SPI	BL
201	Rajesh	8	1
202	Krishna	6	3

- Let *R* be a relation schema having n attributes *A1, A2, A3,…, An*.
- Let M and N be the subsets of attributes of relation R.
- If the values of the M component of a tuple distinctly determine the values of the N component, then there is a functional dependency from M to N.
- It is denoted by M → N
- For e.g., in the above relation RollNo → {Name, SPI,BL}
- It can be also said as N is functionally dependent on M or M functionally determines N.

Functional Dependency

Bank

Account_No	Cust_Name	Balance	Branch
201	Rajesh	80000	Rajkot
202	Krishna	65000	Vadodara

- Find the Functional Dependency.
- Account_No distinctly identifies other attributes. Thus, Account_No → {Cust_Name, Balance, Branch}

Functional Dependency

Types of Functional Dependencies

1. Full Functional Dependency

- In a relation, the attribute B is fully functional dependent on A if B is functionally dependent on A, but not on any proper subset of A.
- For e.g. {Enrollment, Name, Semester} → Result
- We need all three {Enrollment, Name, Semester} to find Result.

Functional Dependency

2. Partial Functional Dependency

- In a relation, the attribute B is partially functional dependent on A if B is functionally dependent on A as well as proper subset of A.
- For e.g. {Enrollment, Name, Semester} → Semester
- If we remove some attributes from A and still the dependency remains as it is, it can said as partial dependency.
- For e.g. we can find the result if we have only Enrollment Number. Thus, {Enrollment} → Semester is a partial dependency.

Functional Dependency

3. Transitive Functional Dependency

- In a relation, if attribute A → B and B → C, then C transitively depends on A.
- For e.g., Account_No → Cust_Name and Cust_Name → Branch, then Account_No → Branch.

Functional Dependency

4. Trivial Functional Dependency

   - Consider A → B and this can be said as Trivial Dependency if B is the subset of A.
   - For e.g. {Enrollment, Name, Semester} → Name

5. Non-Trivial Functional Dependency

   - Consider A → B and this can be said as Non-Trivial Dependency if B is not the subset of A.
   - For e.g. {Enrollment, Name, Semester} → Result

Closure Set : Armstrong's Axioms (Inference Rules)

Armstrong's axioms are a set of rules used to derive the functional dependencies on a relational database.

1. Reflexivity
   If B is a subset of A
   then $A \rightarrow B$

2. Augmentation
   If $A \rightarrow B$
   then $AC \rightarrow BC$

3. Transitivity
   If $A \rightarrow B$ and $B \rightarrow C$
   then $A \rightarrow C$

4. Pseudo Transitivity
   If $A \rightarrow B$ and $BD \rightarrow C$
   then $AD \rightarrow C$

5. Self-determination
   $A \rightarrow A$

6. Decomposition
   If $A \rightarrow BC$
   then $A \rightarrow B$
   and $A \rightarrow C$

7. Union
   If $A \rightarrow B$ and $A \rightarrow C$
   then $A \rightarrow BC$

8. Composition
   If $A \rightarrow B$ and $C \rightarrow D$
   then $AC \rightarrow BD$

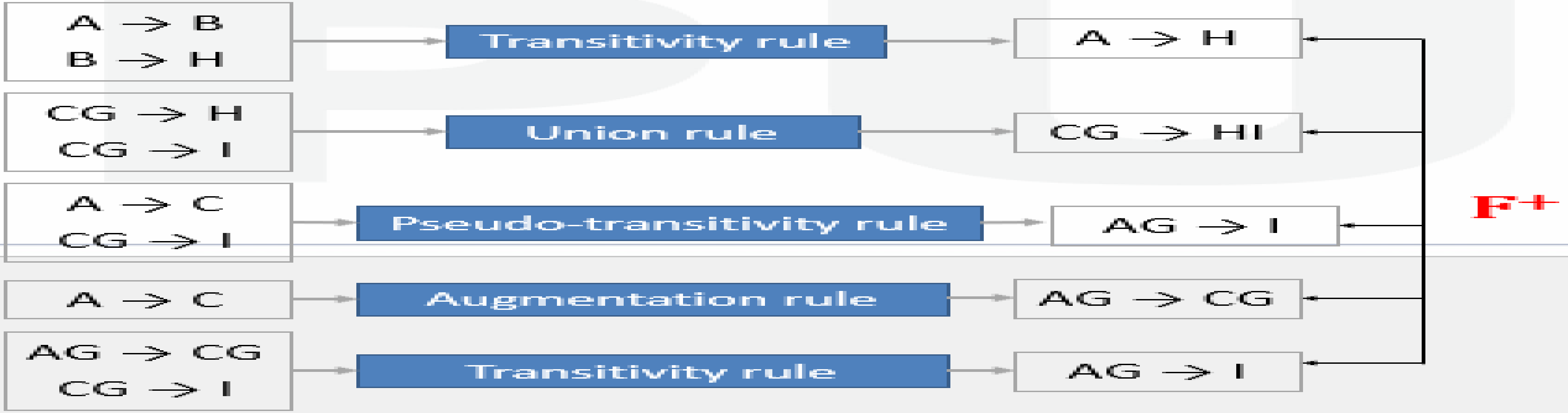
Closure Set of FD (Functional Dependency)

- It is a set of FDs that can be created from an existing set of FD (F).
- For e.g. $F = \{A \rightarrow B, B \rightarrow C\}$, then new FD that can be created is $A \rightarrow C$.
- It is denoted by F^+

Closure Set of FD (Functional Dependency)

Suppose we are given a relation schema R(A,B,C,G,H,I) and the set of functional dependencies are:

$$F = (A \rightarrow B,\ A \rightarrow C,\ CG \rightarrow H,\ CG \rightarrow I,\ B \rightarrow H).\ \text{Find } F^{+}$$



Closure Set of FD (Functional Dependency)

Compute the closure of the following set F of functional dependencies for relational schema R = (A,B,C,D,E,F):

$F = (A \rightarrow B, A \rightarrow C, CD \rightarrow E, CD \rightarrow F, B \rightarrow E)$. **Find F^+**

$A \rightarrow BC$	$A \rightarrow B$ & $A \rightarrow C$	Union Rule
$CD \rightarrow EF$	$CD \rightarrow E$ & $CD \rightarrow F$	Union Rule
$A \rightarrow E$	$A \rightarrow B$ & $B \rightarrow E$	Transitivity Rule
$AD \rightarrow E$	$A \rightarrow C$ & $CD \rightarrow E$	Pseudo-transitivity Rule
$AD \rightarrow F$	$A \rightarrow C$ & $CD \rightarrow F$	Pseudo-transitivity Rule

F^+

$F^+ = \{A \rightarrow BC, CD \rightarrow EF, A \rightarrow E, AD \rightarrow E, AD \rightarrow F\}$

Closure Set Attributes

- The set of all those attributes which can be functionally determined from an attribute set is called as a closure of that attribute set.

- Closure of attribute set {A} is denoted as A^+.

Rules to find Closure set of Attributes:

1. Add the attributes contained in the attribute set for which closure is being calculated to the result set.

2. Recursively add the attributes to the result set which can be functionally determined from the attributes already contained in the result set.

Closure Set Attributes

Consider a relation R (A,B,C,D,E,F,G) with following functional dependencies: $A \rightarrow BC$, $BC \rightarrow DE$, $D \rightarrow F$, $CF \rightarrow G$. Find A^+

Closure of attribute A-

$A^+ = \{ A \}$

$= \{ A , B , C \}$ (Using $A \rightarrow BC$)

$= \{ A , B , C , D , E \}$ (Using $BC \rightarrow DE$)

$= \{ A , B , C , D , E , F \}$ (Using $D \rightarrow F$)

$= \{ A , B , C , D , E , F , G \}$ (Using $CF \rightarrow G$)

Thus,

$A^+ = \{ A , B , C , D , E , F , G \}$

Closure Set Attributes

Consider a relation R (A,B,C,D,E,F,G) with following functional dependencies: A → BC, BC → DE, D → F, CF → G. Find $\{B,C\}^+$

Closure of attribute {B,C} -

$\{ B , C \}^+ = \{ B , C \}$

$= \{ B , C , D , E \}$ (Using BC → DE)

$= \{ B , C , D , E , F \}$ (Using D → F)

$= \{ B , C , D , E , F , G \}$ (Using CF → G)

Thus,

$\{ B , C \}^+ = \{ B , C , D , E , F , G \}$

Parul® University

DIGITAL LEARNING CONTENT

Closure Set Attributes

Consider a relation R (A,B,C,D,E,F,G) with following functional dependencies: $A \rightarrow BC$, $BC \rightarrow DE$, $D \rightarrow F$, $CF \rightarrow G$. Find **D^+**

Closure of attribute D -

$\{ D\}^+ = \{ D\}$

$= \{ D, F\}$ (Using $D \rightarrow F$)

Thus,

$D^+ = \{ D, F\}$, as {D,F} does not determine any other attribute

Closure Set Attributes

Consider a relation R (A,B,C,D,E) with following functional dependencies: $A \rightarrow BC$, $CD \rightarrow E$, $B \rightarrow D$, $E \rightarrow A$.

- Find closure of A.
- Find closure of CD
- Find Closure of B
- Find Closure of BC
- Find closure of E

ANSWER:

$A^+ = ABCDE$

$CD^+ = ABCDE$

$B^+ = BD$

$BC^+ = ABCDE$

$E^+ = ABCDE$

Extraneous Attributes

- Let us consider a relation R with schema R = (A, B, C) and set of functional dependencies **F = { AB → C, A → C }**.

- In **AB → C**, **B is extraneous attribute**, as other FD **A → C**, states that **A alone can determine C**, then **use of B is unwanted**.

- An attribute of a functional dependency is said to be extraneous if we can **remove it without changing the closure of the set of functional dependencies.**

Canonical Cover

- A canonical cover of F is a minimal set of functional dependencies equivalent to F, having no redundant dependencies or redundant parts of dependencies.

- It is denoted by Fc.

- The steps to find canonical cover are:
  - ✓ Use the union rule to replace any dependencies in F
    $A_1 \rightarrow B_1$ and $A_1 \rightarrow B_2$ with $A_1 \rightarrow B_1 B_2$
  - ✓ Find a functional dependency $A \rightarrow B$ with an
    extraneous attribute either in A or in B
  - ✓ If an extraneous attribute is found, delete it from $A \rightarrow B$
    until F does not change
  - ✓ Repeat these steps till F is reduced to Fc.

Canonical Cover (Example)

Consider the relation schema R = (X, Y, Z) with FDs

F = {X → YZ, Y → Z, X → Y, XY → Z} find canonical cover.

- Combine X → YZ and X → Y into X → YZ (Union Rule)
  - ✓ Set is now {X → YZ, Y → Z, XY → Z}
- X is extraneous in XY → Z
  - ✓ Check if the result of deleting X from XY → Z is implied by the other dependencies
    - ❑ Yes: in fact, Y→ Z is already present
  - ✓ Set is now {X → YZ, Y → Z}
- Z is extraneous in X → YZ
  - ✓ Check if X → Z is logically implied by X → Y and the other dependencies
    - ❑ Yes: using transitivity on X → Y and Y → Z.
- The canonical cover is: **X→ Y, Y → Z**

Parul® University

Finding a Candidate Key

Consider a relation R (A,B,C,D,E,F,G,H) with following functional dependencies: **AB → C, A → DE, B → F, F → GH. Find Candidate Key**

AB → C ⟶ A B C D E F G H

A → DE ⟶ A B C D E F G H

B → F ⟶ A B C D E F G H

F → GH ⟶ A B C D E F G H

Find out all the attributes which do not have incoming edge(arrow).

A B C D E F G H

From this, A and B are the attributes with no incoming edge.

- Now, check whether A & B together can identify all other attributes of R. For this, find $(AB)^+$ i.e. Find Closure of AB
- $(AB)^+ = \{A,B,C,D,E,F,G,H\} = R$
- Thus, AB finds the complete Relation R and hence, **AB is a candidate key.**

Finding a Candidate Key

Consider a relation R (A,B,C,D,E) with following functional dependencies:
BC → ADE, D → B. Find Candidate Key

BC → ADE ⟶ A B C D E

D → B ⟶ A B C D E

Find out all the attributes which do not have incoming edge(arrow).

A B C D E

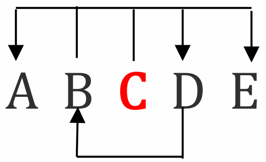
From this, C is the attributes with no incoming edge.

- Now, check whether C can identify all other attributes of R. For this, find C^+ i.e. Find Closure of C
- $C^+ = \{C\}$
- Thus, C alone cannot find all attributes of Relation R, but C is a part of candidate key.

- To Find Candidate Key, Pair C with each remaining attribute of R and find its closure.
- Find $(AC)^+$, $(BC)^+$, $(CD)^+$ and $(CE)^+$

Finding a Candidate Key

Consider a relation R (A,B,C,D,E) with following functional dependencies:
BC → ADE, D → B. Find Candidate Key



- $(AC)^+ = \{A,C\} \neq R$
- $(BC)^+ = \{A,B,C,D,E\} = R$
- $(CD)^+ = \{A,B,C,D,E\} = R$
- $(CE)^+ = \{C,E\} \neq R$
- As (AC) and (CE) do not identify all attributes of Relation R, combine 3 attributes and find its closure as $(CAE)^+$
- $(CAE)^+ = \{C,A,E\} \neq R$
- Thus, the Relation R contains 2 Candidate Keys: **(BC) and (CD).**

Decomposition

- The process of breaking down given relation into two or more relations such that:
    - ✓ Each new relation contains a subset of the attributes of R
    - ✓ All relations must include all tuples and attributes of R
- Types of decomposition
    1. Lossy decomposition
    2. Lossless decomposition (no loss decomposition)

Lossy Decomposition

AcNo	Bal	Branch
001	50000	Rajkot
002	75000	Baroda

AcNo	Bal
001	50000
002	75000

Not Same

Bal	Branch
50000	Rajkot
75000	Baroda

Perform Join

AcNo	Bal	Branch
001	50000	Rajkot
001	75000	Baroda
002	50000	Rajkot

Extra records

- The decomposition of relation R into R1 and R2 is said to be lossy when we join R1 and R2 and still it does not display the same relation as in R.
- Major drawback of this is that we may have data loss.

Lossy Decomposition

AcNo	Bal	Branch
001	50000	Rajkot
002	75000	Baroda

AcNo	Bal
001	50000
002	75000

Same

AcNo	Branch
001	Rajkot
002	Baroda

Perform Join

AcNo	Bal	Branch
001	50000	Rajkot
002	75000	Baroda

Same Table.
No Extra records

- The decomposition of relation R into R1 and R2 is said to be lossless when we join R1 and R2 and it displays the same relation as in R.
- All decomposed relations must be lossless decomposition.

Database Anomalies

- Anomalies means problems and it may occur in the database, if it is designed without proper planning and analysis.

- There can be mainly 3 types of Anomalies:

1. Insert Anomaly
2. Delete Anomaly
3. Update Anomaly

Insert Anomaly

E_No	Ename	Address	D_No	Dname	Dmgr_Id
1	Sagar	Rajkot	1	Sales	1
2	Priya	Baroda	1	Sales	1

- An insert anomaly occurs when inserting certain attribute values is dependent on the presence of values in other attribute.

Consider Employee relation with E_No as a primary key and is referring D_No of Department table, where D_No is also a Primary Key

- Assume that new department "Accounts" has started, but initially there is no employee in that department.
- A user wish to enter the details in Employee relation with no employee details. Hence values of E_No, Ename and Address will be NULL.
- As E_No is Primary Key, it will not allow to enter NULL for it.
- Ultimately, A user will not be able to insert a new record for only department details.

Delete Anomaly

E_No	Ename	Address	D_No	Dname	Dmgr_Id
1	Sagar	Rajkot	1	Sales	1
3	Aakash	Surat	2	Accounts	2

- Delete anomaly occurs when deleting certain attribute values may result in loss of other attribute values.

Consider Employee relation with E_No as a primary key and is referring D_No of Department table, where D_No is also a Primary Key

- Assume that there is only one employee who works in Accounts department and a user wish to delete that record (A Row with E_No = 2).

- Deleting the above Row will also result in no records for department details in the table although we had department details in Department table.

- Ultimately, There are some chances of Data Loss in case of dependencies.

Update Anomaly

E_No	Ename	Address	D_No	Dname	Dmgr_Id
1	Sagar	Rajkot	1	Sales	1
3	Aakash	Surat	2	Accounts	2
2	Priya	Baroda	1	Sales	1

- Update anomaly occurs when updating any records do not reflect changes to all relevant records.

Consider Employee relation with E_No as a primary key and is referring D_No of Department table, where D_No is also a Primary Key

- Assume that the manager of the Sales department is changed. Hence, we need to update Dmgr_Id of all the employees working in Sales department.
- If we fail to update all the corresponding records, i.e. if Dmgr_Id for all the records belonging to Sales department is not updated; then it is possible that different employees may have different Dmgr_Id.

Normal Forms (Normalization)

- In database design, anomalies can be efficiently handled by Normalization.
- Normalization is the process of removing all unwanted as well as redundant data from the table so that database efficiency can be improved in terms of:
  - ✓ Storage (Storing the data using least amount of space)
  - ✓ Scalability (Can work smoothly with constantly growing data)
  - ✓ Data integrity (Consistency and Accuracy of Data)
- It is the process of splitting a single table in to more than one tables such that these tables can be linked using joins whenever a query is executed.
- It is also known as Normal Forms

Normal Forms(Normalization)

Types of Normal Forms

1. 1NF (First normal form)
2. 2NF (Second normal form)
3. 3NF (Third normal form)
4. BCNF (Boyce–Codd normal form)
5. 4NF (Forth normal form)
6. 5NF (Fifth normal form)

As we normalize our database from 1NF to 5NF, overall database complexity and the number of tables increases; but redundancy of database decreases.

Normal Forms (1 NF)

Student

Roll_No	Name	Failed
1	Priya	PPS, EEE
2	Pranav	EME,PPS,EG
3	Romil	PPS,EG
4	Akshar	EG,EME,EEE
5	Nirali	EG,PPS,EME

Multi-Valued Attribute

- A relation is said to be in 1 NF if each attribute has a single value.
- It means that a table must not contain either composite or multi-valued attribute.

- As above relation contains Multi-Valued attribute, it is not in 1NF
- It is quite difficult to find out the students who failed in either PPS and EEE

Normal Forms (1 NF)

Student

Primary Key

Result

Foreign Key

Roll_No	Name
1	Priya
2	Pranav
3	Romil
4	Akshar
5	Nirali

R_Id	Roll_No	Failed
1	1	PPS
2	1	EEE
3	2	EME
4	2	PPS
5	2	EG
…….	…….	………

Splitting a Table into 2 Tables

- The solution to this is to split the table into two tables as following:
  - ✓ One table contains all attributes other than Multi-Valued attributes along with Primary Key attribute
  - ✓ Second table contains multi-valued attribute along with primary key attribute used as Foreign Key.
  - ✓ In this table, Roll_No is a primary key attribute.

Normal Forms (2 NF)

- A table is said to be in 2NF if both the following conditions hold:
  - ✓ Table is in 1st Normal Form
  - ✓ No non-prime attribute is dependent on the proper subset of any candidate key of table.
- **An attribute that is not part of any candidate key is known as non-prime attribute.**

School

teacher_id	subject	teacher_age
111	Maths	38
111	Physics	38
222	Biology	38
333	Physics	40
333	Chemistry	40

- Since a teacher can teach more than one subjects, the table can have multiple rows for a same teacher.
- Here, Candidate Keys are: **{teacher_id,subject}** and non-prime attributes are: **{teacher_age}**

Normal Forms (2 NF)

Teachers

teacher_id	teacher_age
111	38
222	38
333	40

Candidate Keys:
{teacher_id}
Non-Prime Attributes:
{teacher_age}

Subjects

teacher_id	subject
111	Maths
111	Physics
222	Biology
333	Physics
333	Chemistry

Candidate Keys:
{teacher_id,subject}

- A table is in 1NF as all attributes has single values.
- A table is not in 2NF as teacher_age is fully dependent on teacher_id and teacher_id is proper subset of Candidate Key.
- **This rule gets violated: No non-prime attribute is dependent on the proper subset of any candidate key of table.**
- The solution for this would be splitting it into more than one table such that it satisfies both the conditions.

Splitting a Table into 2 Tables

Normal Forms (3 NF)

- A table is said to be in 3NF if both the following conditions hold:
  - ✓ Table is in 2nd Normal Form
  - ✓ Transitive functional dependency of non-prime attribute on any super key should be removed.
- **An attribute that is not part of any candidate key is known as non-prime attribute.**
- A table is in 3NF if it is in 2NF and for each functional dependency X-> Y at least one of the following conditions hold:
  - ✓ X is a Super Key
  - ✓ Y is a Prime Attribute
- **An attribute that is a part of any candidate key is known as prime attribute.**

Normal Forms (3 NF)

Employee

id	name	zip	state	city	district
1001	John	282005	UP	Agra	Dayal Bagh
1002	Ajeet	222008	TN	Chennai	M-City
1006	Lora	282007	TN	Chennai	Urrapakkam
1101	Lilly	292008	UK	Pauri	Bhagwan
1201	Steve	222999	MP	Gwalior	Ratan

- ✓ **Super keys**: {id}, {id, name}, {id, name, zip}…so on
- ✓ **Candidate Keys**: {id}
- ✓ **Non-prime attributes**: all attributes except {id} are non-prime as they are not part of any candidate keys.

{zip} → {state, city, district}
{id} → {zip}
So transitively, {id} → {state, city, district}

Here, {state, city, district} are Non-Prime attributes and are functionally dependent on Candidate Key {id}. Hence it violates rule of 3NF

Normal Forms (3 NF)

- The solution for this would be splitting it into more than one table such that transitive dependency is removed

id	name	zip
1001	John	282005
1002	Ajeet	222008
1006	Lora	282007
1101	Lilly	292008
1201	Steve	222999

Employee_Details

Employee_Address

zip	state	city	district
282005	UP	Agra	Dayal Bagh
222008	TN	Chennai	M-City
282007	TN	Chennai	Urrapakkam
292008	UK	Pauri	Bhagwan
222999	MP	Gwalior	Ratan

Normal Forms Boyce-Code NF (BCNF)

A relation is said to be in BCNF if It is in 3NF and every determinant should be primary key.

BCNF is based on the concept of a determinant.

Primary Key

Determinant

Dependent

Account_No → {Balance, Branch}

Account_No → {Balance, Branch}

Normal Forms Boyce-Code NF (BCNF)

Student	Subject	Faculty
Anand	DBMS	Mehta
Rohan	ETC	Vora
Ankit	DBMS	Patel
Riya	ETC	Joshi
Nisha	ETC	Vora
Rohan	DBMS	Mehta
Anand	ETC	Joshi
Nisha	DBMS	Patel

- ✓ FD1: Student, Subject → Faculty
- ✓ FD2: Faculty → Subject
- ✓ So {Student, Subject} → Subject (Using Transitivity rule)

- ➢ Here, One faculty teaches only one subject.
- ➢ But, one subject is taught by more than one faculty.
- ➢ In FD2, Faculty is determinant, but it is not Primary Key.
- ➢ Hence, the relation is not in BCNF

Normal Forms Boyce-Code NF (BCNF)

- In the above relation, one can student may more than one assignments with different faculties.
- Thus, the data will be stored repeatedly for student, subject and faculty.
- This will lead to the requirement of more storage space.

Normal Forms Boyce-Code NF (BCNF)

Faculty_Subject

Faculty	Subject
Mehta	DBMS
Vora	ETC
Patel	DBMS
Joshi	ETC
Vora	ETC
Mehta	DBMS
Joshi	ETC
Patel	DBMS

Faculty_Student

Student	Faculty
Anand	Mehta
Rohan	Vora
Ankit	Patel
Riya	Joshi
Nisha	Vora
Rohan	Mehta
Anand	Joshi
Nisha	Patel

- The solution for this would be splitting it into more than one table such that transitive dependency is removed and every determinant is a Primary Key

Normal Forms (4NF)

X Y

Id	Subject	Project
1	DBMS	P1
1	ETC	P1
1	DBMS	P2
1	ETC	P2

- A relation R is in fourth normal form (4NF) if and only if it is in BCNF and has no multivalued dependencies.
- Multivalued Dependency: For a dependency X → Y, if for a single value of X, there exists multiple values of Y, then the table may have multi-valued dependency.

- Multivalued dependency (MVD) is denoted by →→
- The above Multivalued dependency (MVD) can be represented as X →→ Y

Parul® University

DIGITAL LEARNING CONTENT

Normal Forms (4NF)

Id	Subject	Project
1	DBMS	P1
1	ETC	P1
1	DBMS	P2
1	ETC	P2

Decompose

Id	Subject
1	DBMS
1	ETC

Id	Activity
1	P1
1	P2

A table can have both functional as well as multivalued dependency.

Id	Address	Subject	Project
1	Amin Marg	DBMS	P1
1	Amin Marg	ETC	P1
1	Amin Marg	DBMS	P2
1	Amin Marg	ETC	P2

- ✓ $Id \rightarrow Address$
- ✓ $Id \rightarrow\rightarrow Subject$
- ✓ $Id \rightarrow\rightarrow Project$

Normal Forms (4NF)

Id	Address	Subject	Project
1	Amin Marg	DBMS	P1
1	Amin Marg	ETC	P1
1	Amin Marg	DBMS	P2
1	Amin Marg	ETC	P2

Decompose

Id	Address
1	Amin Marg
1	Amin Marg
1	Amin Marg
1	Amin Marg

Id	Subject
1	DBMS
1	ETC
1	DBMS
1	ETC

Id	Project
1	P1
1	P1
1	P2
1	P2

Normal Forms (5NF)

SUBJECT	LECTURER	SEMESTER
Computer	Anshika	Semester 1
Computer	John	Semester 1
Math	John	Semester 1
Math	Akash	Semester 2
Chemistry	Praveen	Semester 1

- A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.
- 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.
- 5NF is also known as Project-join normal form (PJ/NF).

- In the above table, John takes both Computer and Math class for Semester 1 but he doesn't take Math class for Semester 2. In this case, combination of all these fields required to identify a valid data.
- Suppose we add a new Semester as Semester 3, but do not know about the subject and who will be taking that subject. So we leave Lecturer and Subject as NULL. But as all three columns acts as a primary key, we can't leave other two columns blank.

Normal Forms (5NF)

- The solution for this would be splitting a relation into multiple tables such that non lossless decomposition is maintained.

R1

SEMESTER	SUBJECT
Semester 1	Computer
Semester 1	Math
Semester 1	Chemistry
Semester 2	Math

R2

SUBJECT	LECTURER
Computer	Anshika
Computer	John
Math	John
Math	Akash
Chemistry	Praveen

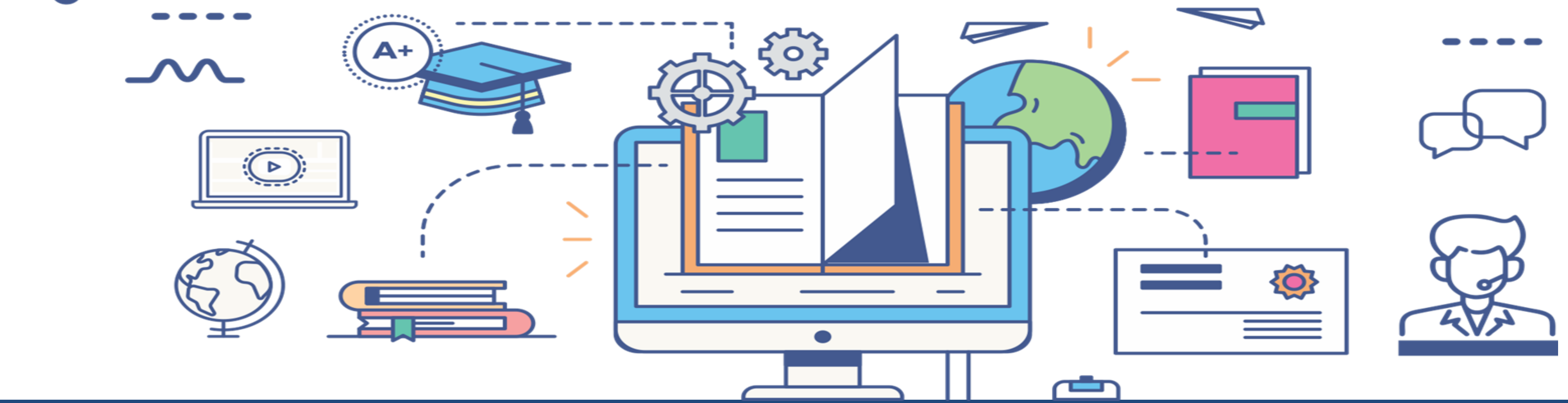
R3

SEMSTER	LECTURER
Semester 1	Anshika
Semester 1	John
Semester 1	John
Semester 2	Akash
Semester 1	Praveen

Normal Forms (Special Note)

- A database is never normalized to 5th NF.
- Once the database is converted to BCNF, it is assumed that all the redundancies of the table has been removed.
- A database is converted to 4th NF and 5th NF only if the database administrator doubts the presence of redundant data in the database.
- In real world, the database is created by considering all aspects like data duplication, storage size, data access and many more.
- Thus, Normalization of database may or may not be required.

DIGITAL LEARNING CONTENT



Parul® University